



Building Java Programs
A BACK TO BASICS APPROACH 5e

Fundamentals of Computer Science

II

09 - Tree (Basic)

Instructor: Varik Hoang



Objectives

- Binary Tree Basics
- Tree Traversals
- Common Tree Operations
- Binary Search Trees



Binary Tree - Height Implementation

```
public class BinaryTree
{
    private TreeNode root;
    ...
    public void height()
    {
        height(root) - 1; // count edges instead of nodes
    }

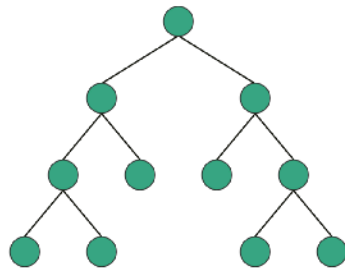
    private void height(TreeNode current)
    {
        // base case
        if (current == null)
            return 0;

        // recursive case
        int left = height(current.left);
        int right = height(current.right);
        return 1 + (left < right ? right : left); // count 1 for current
    }

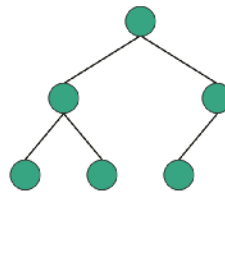
    class TreeNode { ... }
}
```

Full & Complete & Balanced Binary Tree

- **Full:** all the nodes have **no child** or **two children**
- **Complete:** all the levels are **completely filled** except **possibly the lowest one**, which is filled from the left
- **Balanced:** One whose **subtrees differ** in height by **at most 1** and balanced themselves
- **Perfect:** every node has **exact two children** and **all leaf nodes** are at the **same level**
- **Degenerate:** every node has **one single child** even left or right
- **Skewed:** is a **pathological/degenerate** tree either dominated by the left nodes or the right nodes



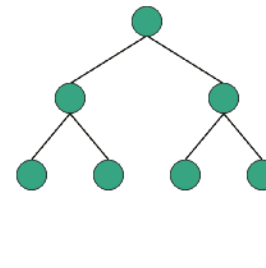
Full



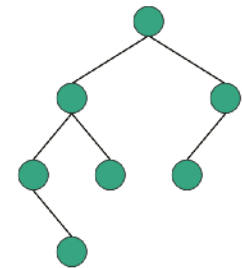
Complete



Degenerate



Perfect

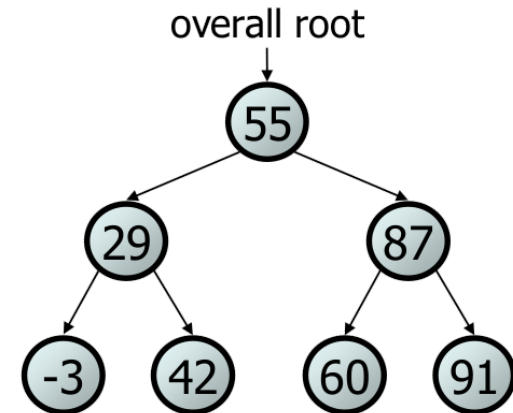


Balanced



Binary Search Tree (BST)

- Binary search tree (BST) is a binary tree that is either
 - Empty
 - A node R such that
 - Elements of R's left subtree has value less than R's data
 - Elements of R's right subtree has value greater than R's data.
 - BSTs store their elements in sorted order





Generic Node

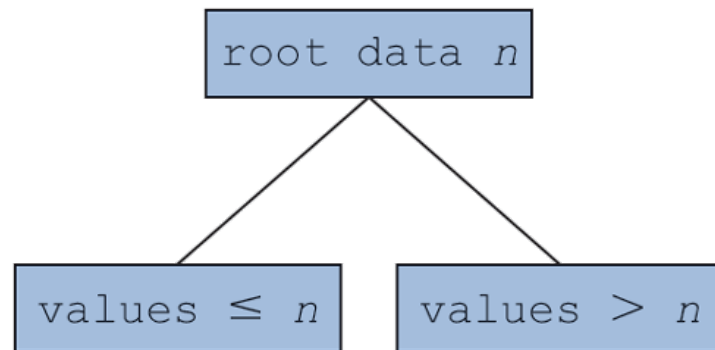
- A single element of a structure such as a BST
- Each node contains:
 - one data *value*
 - one *reference* to the *left* node
 - one *reference* to the *right* node

```
class TreeNode<Type>
{
    Type data;
    TreeNode<Type> left;
    TreeNode<Type> right;
}
...
TreeNode myNode = new TreeNode<Type>();
myNode.data = 18;
myNode.left = null;
myNode.right = null;
```



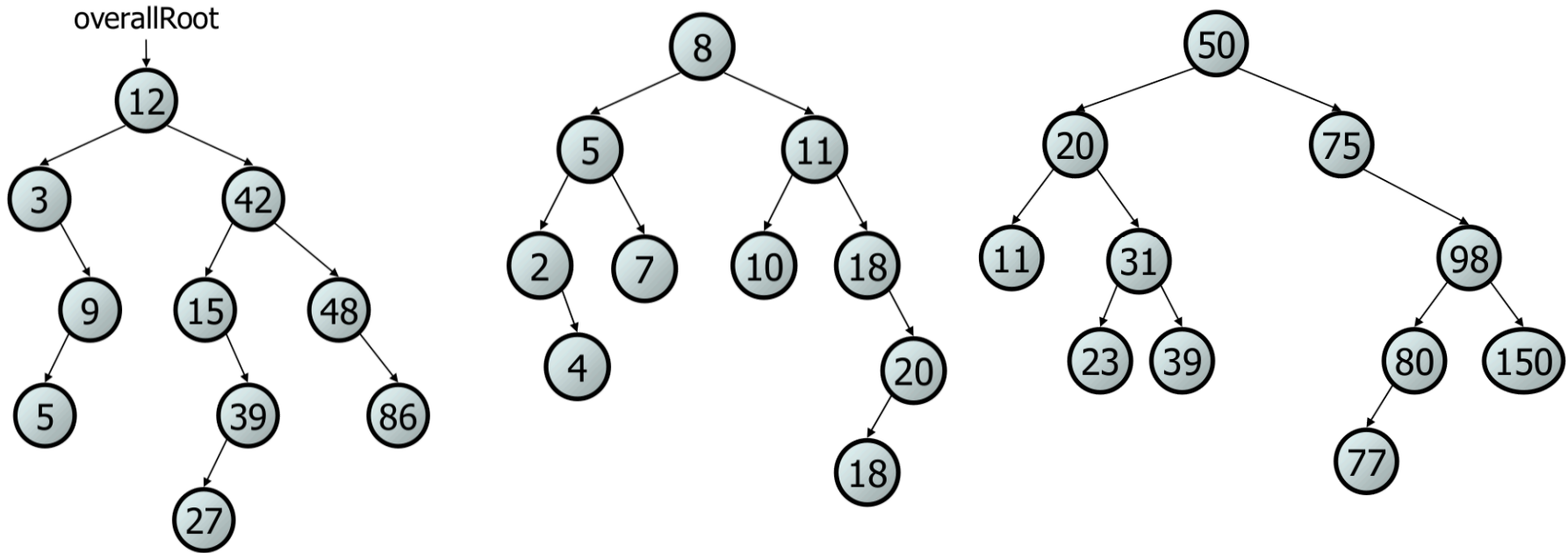
Binary Search Tree (BST)

- A binary search tree is a structural parallel to the binary search algorithm
- The algorithm can efficiently search through subtrees divided the data in half
 - Left subtree contains items which have values $\leq$ the value of current node
 - Right subtree contains items which have values $>$ the value of current node
- Sometimes we want to allow duplicates:
 - The duplicated value appears on the left subtree
 - Do nothing since the value is already in the tree
 - Increase the counter variable in the node





BST Search





BST Class

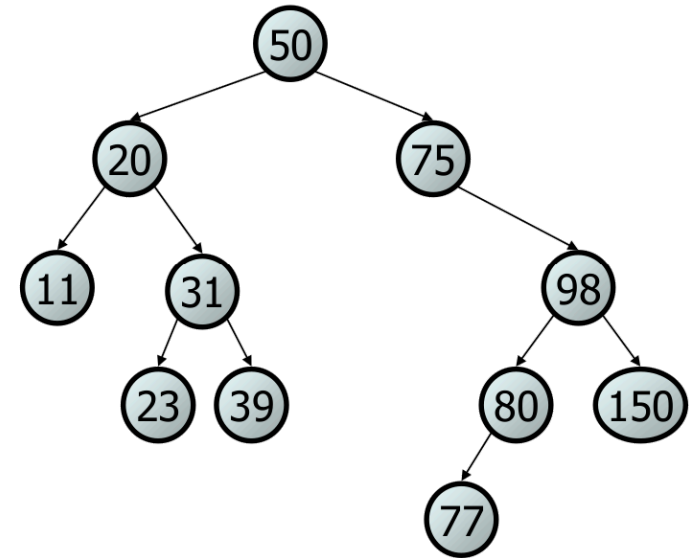
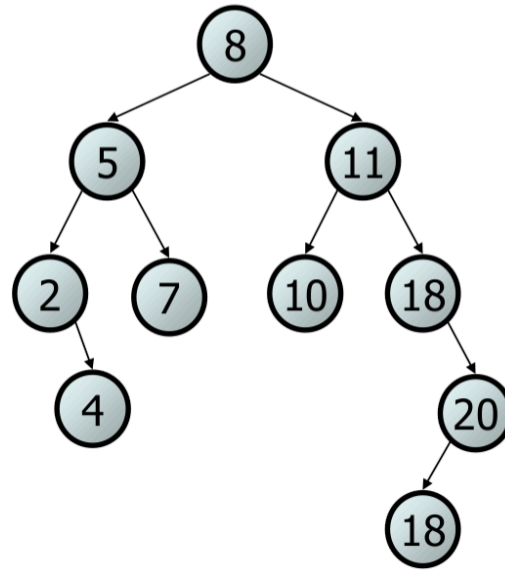
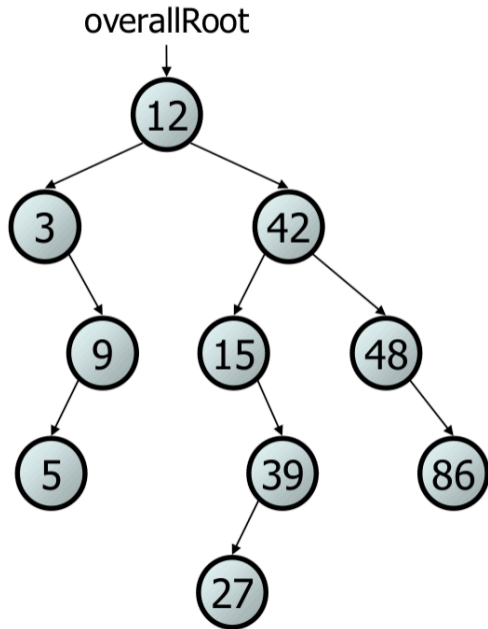
```
public class BST<Type extends Comparable<Type>>
{
    private TreeNode<Type> root;

    public BST()
    {
        this.root = null;
    }

    class TreeNode { ... }
}
```



BST Add





BST Class - Add Method Implementation

```

public class BST<Type extends Comparable<Type>>
{
    ...
    public void add(Type value)
    {
        add(value, root);
    }

    private TreeNode<Type> add(Type value, TreeNode<Type> current)
    {
        if (current == null) // base case
            current = new TreeNode<Type>(value);
        else if (value.compareTo(current.data) < 0) // recursive case
            add(value, current.left);
        else
            add(value, current.right);
        return current;
    }

    class TreeNode { ... }
}

```

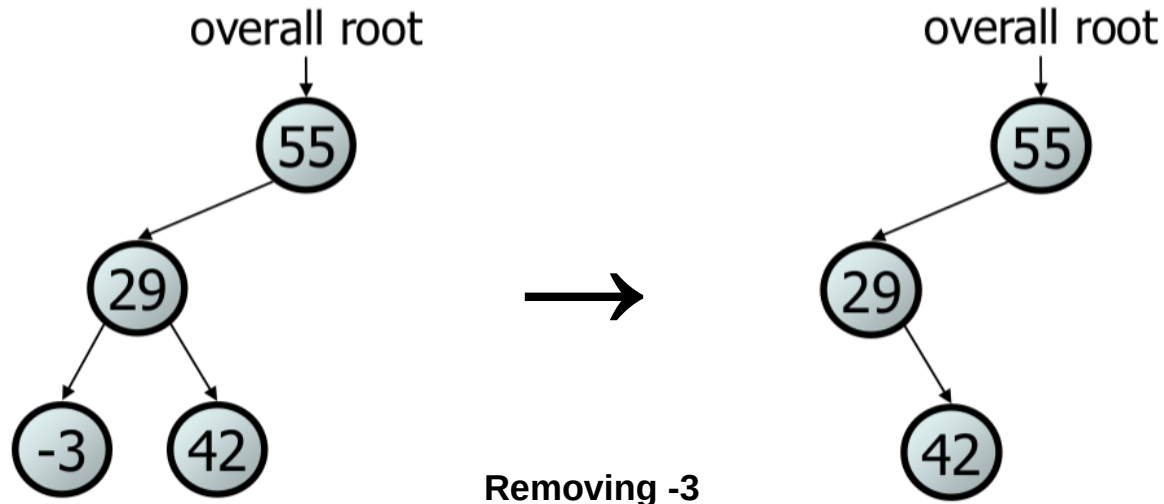


BST Class - Remove Method

- Remove leaf (with no child)
- Remove node with only left child
- Remove node with only right child
- Remove node with both children

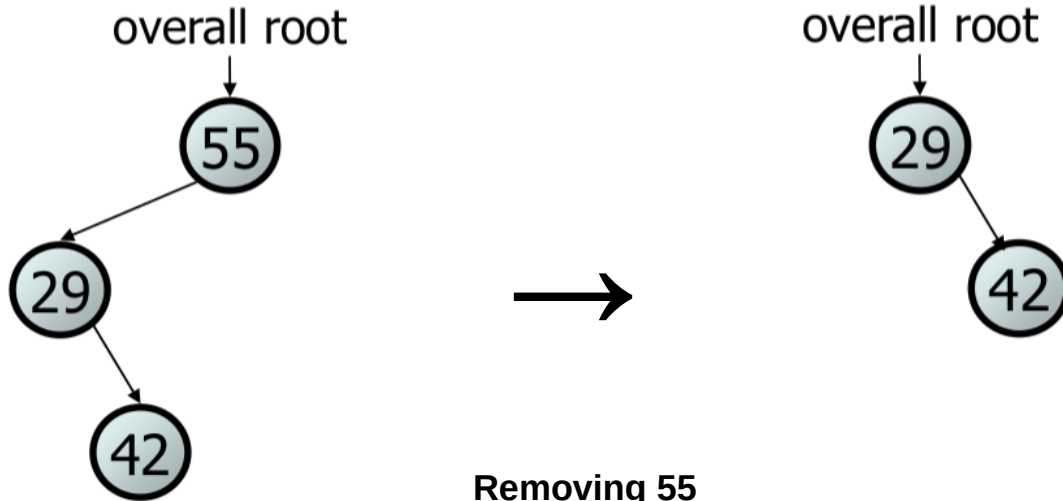
BST Class - Remove Method (cont.)

- **Remove leaf (with no child)**
- Remove node with only left child
- Remove node with only right child
- Remove node with both children



BST Class - Remove Method (cont.)

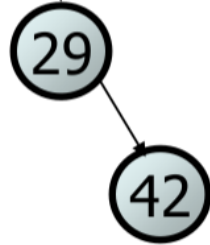
- Remove leaf (with no child)
- **Remove node with only left child**
- Remove node with only right child
- Remove node with both children



BST Class - Remove Method (cont.)

- Remove leaf (with no child)
- Remove node with only left child
- **Remove node with only right child**
- Remove node with both children

overall root



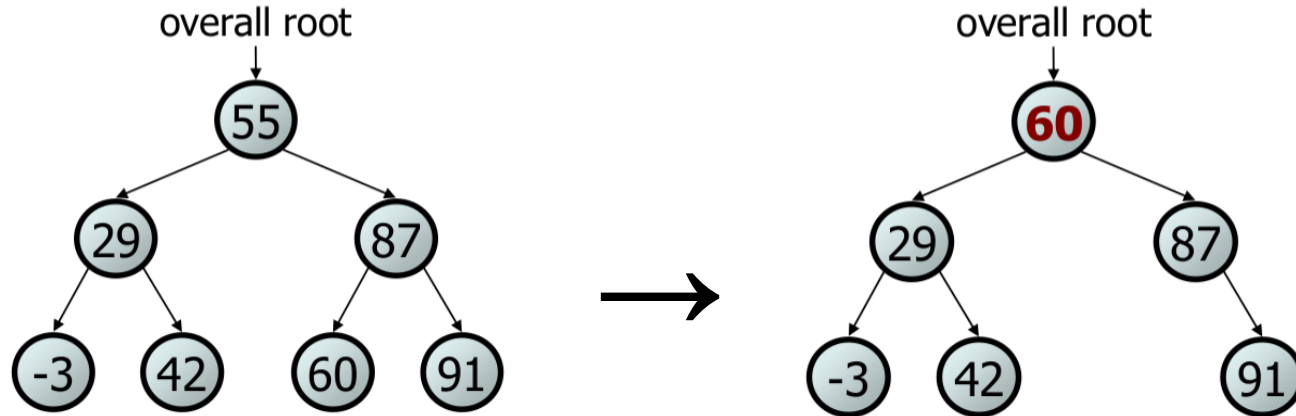
overall root



Removing 29

BST Class - Remove Method (cont.)

- Remove leaf (with no child)
- Remove node with only left child
- Remove node with only right child
- **Remove node with both children**



Removing 55



BST Class - Remove Method Implementation

```
...
private TreeNode<Type> remove(Type value, TreeNode<Type> current)
{
    if (current == null) // base case
        return null;

    if (value.compareTo(current.data) < 0) // recursively go down
        current.left = remove(value, current.left);
    else if current.right = remove(value, current.right);
    else
    {
        if (root.left == null) // node with only right child
            return root.right;
        else if (root.right == null) // node with only left child
            return root.left;

        // node with two children -> find in-order successor
        root.data = findMin(root.right);
        // delete the in-order successor
        root.right = remove(root.right, root.data);
    }
    return current;
}
...
```

```
...
public void remove(Type value)
{
    root = remove(value, root);
}

private Type findMin(TreeNode current)
{
    Type min = current.data;
    while (current.left != null)
    {
        min = current.left.data;
        current = current.left;
    }

    return min;
}
...
```

Questions & Answer



Thank You!

